

Fondamenti di Informatica

A.A. 2018-2019

Prova intermedia del 14 giugno 2019 – Testo 1

NOME _____ COGNOME _____

Durante l'esame non si possono usare appunti né altro materiale.

Esercizio 1

Si dica se ciascuna delle seguenti affermazioni relative ai *metodi di una classe C++* è corretta:

Sì No

- | | | |
|--------------------------|--------------------------|--|
| ☐ | ☐ | I selettori non possono essere implementati come metodi costanti. |
| ☐ | ☐ | I modificatori accedono agli attributi di un oggetto e ne modificano il valore. |
| ☐ | ☐ | I costruttori possono essere dotati di una lista di inizializzazione. |
| ☐ | ☐ | Il distruttore non ha valore di ritorno e non può avere parametri. |
| ☐ | ☐ | Tra gli operatori sovraccaricati deve sempre figurare l'operatore di assegnamento. |
| ☐ | ☐ | Il distruttore viene chiamato automaticamente quando si distrugge un oggetto della classe. |
| ☐ | ☐ | Le funzioni membro che sovraccaricano operatori unari hanno un parametro. |
| ☐ | ☐ | I costruttori non hanno valore di ritorno e non possono avere parametri. |
| ☐ | ☐ | Tra i metodi di una classe, hanno una particolare rilevanza le funzioni amiche. |
| ☐ | ☐ | Gli operatori sovraccaricati implementano per la classe gli operatori standard del C++. |

Esercizio 2

Individuare e correggere gli errori nel seguente testo, riguardante le *liste concatenate*:

Una lista concatenata è una sequenza di elementi (detti anche oggetti) collegati tra loro da puntatori. Una lista concatenata può contenere un numero qualunque di elementi (a meno delle dimensioni della memoria del calcolatore) e può crescere o ridursi a seconda delle esigenze, grazie all'utilizzo della allocazione statica. Le liste concatenate si possono distinguere in quattro categorie principali: liste semplici nelle quali ogni elemento contiene un puntatore all'elemento precedente, liste doppiamente concatenate nelle quali ogni elemento contiene un puntatore all'elemento successivo e un puntatore a quello precedente, liste circolari semplici nelle quali l'ultimo elemento contiene un puntatore al primo elemento e liste circolari doppiamente concatenate nelle quali l'ultimo elemento contiene un puntatore al primo elemento e il primo un puntatore all'ultimo. Nelle liste semplici gli elementi si compongono di due parti, l'informazione e il puntatore all'elemento successivo, la lista inizia con un puntatore al suo ultimo elemento e termina con un puntatore nullo. Tra le operazioni applicabili alle liste si possono enunciare le seguenti: creazione, distruzione, inserimento di nuovi elementi (in testa, in coda oppure in mezzo alla lista), rimozione di elementi (in testa, in coda oppure in mezzo alla lista), attraversamento della lista applicando specifiche operazioni a ciascun elemento.

Soluzione:

Esercizio 3

Completare le seguenti affermazioni:

1. Tramite il puntatore _____, ciascun oggetto ha accesso al proprio indirizzo di memoria.
2. Una classe si dice _____ se contiene attributi che sono a loro volta oggetti di una classe.
3. I _____ forniscono un meccanismo per definire classi generiche.
4. Una funzione può essere chiamata in modo indiretto tramite un _____.

Esercizio 4

Si scriva il valore stampato a video dai seguenti programmi C++:

Attenzione: è richiesto di riportare brevemente i passaggi seguiti per ottenere la soluzione.

```
1 int test(int a[], int n) {  
    if (n <= 0)  
        return 0;  
    else  
        return a[n - 1] + test(a, n - 1);  
}  
  
int main() {  
    int v[5] = {2, 3, 1, 6, 8};  
    cout << test(v, 5) << endl;  
    return 0;  
}
```

```
2 int main() {  
    double w[3] = {1.2, 3.0, 2.8};  
    double x = w[0];  
    double& y = w[1];  
    double* p = &w[2];  
    double* q = p - 2;  
    y = *p + *q;  
    q = p;  
    double t = (x + *q) - w[1];  
    cout << t << endl;  
    return 0;  
}
```

Soluzione:

Fondamenti di Informatica

A.A. 2018-2019

Prova intermedia del 14 giugno 2019

Gli esercizi 5 e 6, qui sotto, dovranno essere svolti al calcolatore e consegnati secondo le seguenti modalità:

- **Compilare il modulo che compare nella finestra che avete trovata aperta sul vostro PC. Se non la trovate, fate doppio click sull'icona Esame che compare sul desktop.**
 - **Creare un file .zip contenente il codice sorgente (cioè i file .cpp), caricarlo e inviarlo.**
-

Esercizio 5

Si scriva il template di funzione *Sottocampiona* che riceva come parametri un array a di elementi di tipo generico T , la sua dimensione n (un numero intero positivo) e un secondo numero intero m tale che $0 < m \leq n / 2$. Il template di funzione restituisce come valore di ritorno un puntatore a un array di elementi di tipo generico T e opera in questo modo: calcola $k = \lceil n/m \rceil$ (cioè il quoziente della divisione di n per m , approssimato al numero intero immediatamente superiore), alloca dinamicamente un array s di m elementi di tipo generico T , copia in s un elemento ogni k dell'array a , partendo dal primo elemento di a . Ad esempio, assumendo che T venga istanziato con il tipo concreto *int* e che siano dati $a = \{3, 2, 8, 5, 4, 9, 6, 1, 2, 1\}$ ($n = 10$) e $m = 3$, il template di funzione allocherà un array s di 3 elementi e lo riempirà copiandovi un elemento ogni $k = \lceil 10/3 \rceil = 4$ elementi di a (copierà quindi il primo, il quinto e il nono elemento di a), cioè si avrà $s = \{3, 4, 2\}$. Se il valore di n o m non è valido, il template di funzione restituisce NULL. Si supponga che il template di funzione venga istanziato con un tipo concreto dotato di operatore di assegnamento.

Si scriva quindi un programma C++ che operi come segue: dichiarare un array v di 10 numeri interi e chieda all'utente di inserirne i valori da tastiera, chieda all'utente di inserire un numero intero t compreso tra 1 e 5 (in caso di inserimento di un valore non valido, il programma chiederà di ripetere l'inserimento finché non pervenga un valore valido), chiami il template di funzione *Sottocampiona* passando l'array v , la sua dimensione e la dimensione del nuovo array t come parametri, stampi a video i valori contenuti nell'array puntato dal puntatore che il template di funzione restituisce come valore di ritorno (tale array avrà dimensione t), deallochi tale array e termini.

Esercizio 6

Allo scopo di gestire il magazzino di un grande negozio di scarpe, si sviluppi in C++ la classe *Scarpa* avente i seguenti attributi: un codice identificativo della marca e del modello (un numero intero), il numero della scarpa (un numero intero), la quantità di scarpe presenti in magazzino (un numero intero) e il prezzo in Euro della scarpa (un numero reale). Si implementino, inoltre, i seguenti metodi:

- Il costruttore di default che inizializzi a zero tutti gli attributi, usando la lista di inizializzazione.
- Un costruttore con parametri che riceva come parametri i valori del codice identificativo di marca e modello, del numero della scarpa, della quantità di scarpe presenti in magazzino e del prezzo e inizializzi i corrispondenti attributi della classe usando la lista di inizializzazione. Si assuma che i valori dei parametri siano sempre validi.
- Il distruttore.
- I selettori (un metodo per ciascun attributo).
- I modificatori (un metodo per ciascun attributo). Per semplicità, si assuma che i valori assegnati ai parametri dei modificatori siano sempre validi.
- L'operatore $\leq$ per il quale una scarpa è minore o uguale di un'altra se il numero della scarpa è minore o uguale del numero dell'altra scarpa.
- L'operatore di assegnamento.
- L'operatore di estrazione.

Si sviluppi quindi la funzione *main* che operi come segue:

- Apra in lettura il file *MagazzinoScarpe.txt*. Tale file contiene nella sua prima riga il numero di righe n di cui il file si compone e nelle righe successive, per ciascuna di esse, le informazioni relative alle scarpe nel seguente ordine: codice identificativo di marca e modello, numero, quantità di scarpe presenti in magazzino, prezzo.
- Legga la prima riga del file *MagazzinoScarpe.txt* e allochi dinamicamente un array ms di n oggetti di tipo *Scarpa*.
- Sfruttando l'operatore di estrazione, sopra definito per la classe *Scarpa*, legga le righe successive del file e assegni agli elementi dell'array ms le informazioni contenute in ciascuna riga del file. Per semplicità, si supponga che il file non sia vuoto e che i dati in esso contenuti siano validi.
- Chieda all'utente di inserire da tastiera un numero di scarpa, scandisca l'array ms e stampi a video la quantità di scarpe presenti in magazzino per il numero inserito dall'utente.
- Deallochi l'array ms , chiuda il file e termini.

Soluzione

Esercizio 1 – Soluzione (1.5 punti, 0.15 per ogni risposta esatta)

No, Sì, Sì, Sì, No, Sì, No, No, No, Sì.

Esercizio 2– Soluzione (1.0 punto, 0.25 per ogni risposta esatta)

Una lista concatenata è una sequenza di elementi (detti anche **nodi**) collegati tra loro da puntatori. Una lista concatenata può contenere un numero qualunque di elementi (a meno delle dimensioni della memoria del calcolatore) e può crescere o ridursi a seconda delle esigenze, grazie all'utilizzo della allocazione **dinamica**. Le liste concatenate si possono distinguere in quattro categorie principali: liste semplici nelle quali ogni elemento contiene un puntatore all'elemento **successivo**, liste doppiamente concatenate nelle quali ogni elemento contiene un puntatore all'elemento successivo e un puntatore a quello precedente, liste circolari semplici nelle quali l'ultimo elemento contiene un puntatore al primo elemento e liste circolari doppiamente concatenate nelle quali l'ultimo elemento contiene un puntatore al primo elemento e il primo un puntatore all'ultimo. Nelle liste semplici gli elementi si compongono di due parti, l'informazione e il puntatore all'elemento successivo, la lista inizia con un puntatore al suo **primo** elemento e termina con un puntatore nullo. Tra le operazioni applicabili alle liste si possono enunciare le seguenti: creazione, distruzione, inserimento di nuovi elementi (in testa, in coda oppure in mezzo alla lista), rimozione di elementi (in testa, in coda oppure in mezzo alla lista), attraversamento della lista applicando specifiche operazioni a ciascun elemento.

Esercizio 3 – Soluzione (1.0 punto, 0.25 per ogni risposta esatta)

This, composta, template di classe, puntatore a funzione.

Esercizio 4 – Soluzione (1.5 punto, 0.75 per ogni risposta esatta)

1. Il programma stampa 20; 2. Il programma stampa 0.

Esercizio 5 – Soluzione (2.0 punti, 1.0 – 1.0)

```
#include <iostream>
using namespace std;

template <typename T>
T* Sottocampiona(T a[], int n, int m) {
    if (n < 0 || m <= 0 || m > n / 2)
        return NULL;
    int k = n / m;
    if (n % m != 0)
        k += 1;
    T* s = new T[m];
    if (s == NULL) return NULL;
    for (int i = 0; i < n; i += k)
        s[i / k] += a[i];
    return s; }

int main() {
    const int dim = 10;
    int v[dim] = {0};

    cout << "Inserire " << dim << " numeri interi: " << endl;
    for (int i = 0; i < dim; i++)
        cin >> v[i];
    int t = 0;
    cout << "Inserire un numero tra 1 e " << dim / 2 << endl;
    do {
        cin >> t;
        if (t <= 0 || t > dim / 2)
            cout << "Valore non valido" << endl;
    } while (t <= 0 || t > dim / 2);
    int* p = Sottocampiona(v, dim, t);
    if (p != NULL) {
        for (int i = 0; i < t; i++)
            cout << p[i] << " ";
        cout << endl;
        delete[] p; }
    return 0; }
```

Esercizio 6 – Soluzione (4.0 punti, 0.8 – 0.2 – 0.2 – 0.15 – 0.15 – 0.15 – 0.15 – 0.15 – 0.15 – 0.15 – 0.35 – 0.35 – 0.35 – 0.4)

```
#include <iostream>
#include <fstream>
using namespace std;

class Scarpa {
public:
    Scarpa();
    Scarpa(int id, int n, int q, double p);
    ~Scarpa();
    int getID() const;
    int getNumero() const;
    int getQuantità() const;
    double getPrezzo() const;
    void setID(int id);
    void setNumero(int n);
    void setQuantità(int q);
    void setPrezzo(double p);
    int operator<=(Scarpa s) const;
    const Scarpa& operator=(const Scarpa& s);
    friend istream& operator>>(istream& is, Scarpa& s);
private:
    int _id;
    int _numero;
    int _quantità;
    double _prezzo; };

Scarpa::Scarpa()
: _id(0), _numero(0), _quantità(0), _prezzo(0.0) {}

Scarpa::Scarpa(int id, int n, int q, double p)
: _id(id), _numero(n), _quantità(q), _prezzo(p) {}

Scarpa::~Scarpa(){}

int Scarpa::getID() const { return _id; }
int Scarpa::getNumero() const { return _numero; }
int Scarpa::getQuantità() const { return _quantità; }
double Scarpa::getPrezzo() const { return _prezzo; }

void Scarpa::setID(int id) { _id = id; }
void Scarpa::setNumero(int n) { _numero = n; }
void Scarpa::setQuantità(int q) { _quantità = q; }
void Scarpa::setPrezzo(double p) { _prezzo = p; }

int Scarpa::operator<=(Scarpa s) const {
    if (_numero <= s._numero) return 1;
    else return 0; }

const Scarpa& Scarpa::operator=(const Scarpa& s) {
    if (&s != this) {
        _id = s._id; _numero = s._numero;
        _quantità = s._quantità; _prezzo = s._prezzo; }
    return *this; }

istream& operator>>(istream& is, Scarpa& s) {
    is>>s._id>>s._numero>>s._quantità>>s._prezzo;
    return is; }

int main() {
    fstream fin;
    fin.open("MagazzinoScarpe.txt", ios::in);
    int n = 0;
    fin >> n;
    Scarpa* ms = new Scarpa[n];
    if (ms == NULL) return -1;
    for (int i = 0; i < n; i++)
        fin >> ms[i];
    int contatore = 0; int num = 0;
    cout << "Inserire un numero di scarpa: ";
    cin >> num;
    for (int k = 0; k < n; k++)
        if (ms[k].getNumero() == num)
            contatore += ms[k].getQuantità();
    cout << "Quantita': " << contatore << endl;
    fin.close();
    delete[] ms;
    return 0; }
```